Gobierno Bolivariano de Venezuela | Ministerio del Poder Popular para la Educación Universitaria

Universidad Politécnica Territorial de Perú "Luis Mariano Rivera"

DIVISIÓN DE SERVICIO COMUNITARIO U.P.T.P. "LUIS MARIANO RIVERA"

1816-2016 PATRIOTAS UNIDOS SABEMOS VENCER

Manual Técnico

Manual Técnico: Sistema de Inscripción Escolar Estadístico

El Manual Técnico proporciona información detallada sobre la arquitectura, configuración y funcionamiento interno del Sistema de Inscripción Escolar Estadístico. Está diseñado para desarrolladores, administradores de sistemas y cualquier persona interesada en comprender cómo se construyó y cómo se puede mantener y mejorar en el futuro.

1. Arquitectura del Sistema:

- El Sistema de Inscripción Escolar Estadístico sigue una arquitectura basada en tres capas: presentación, lógica y persistencia de datos.
- La capa de presentación utiliza tecnologías web como HTML5, CSS3, PHP y Java Script para proporcionar una interfaz de usuario intuitiva y receptiva.
- La capa de lógica de negocio contiene la lógica empresarial y reglas del sistema, implementada principalmente en Java Script y PHP.
- La capa de persistencia utiliza una base de datos relacional (por ejemplo, MySQL) para almacenar y recuperar los datos del sistema.

2. Requisitos de Software:

- Sistema Operativo: Se recomienda un sistema operativo compatible con los lenguajes y tecnologías utilizadas, como Windows, macOS o Linux.
- Servidor Web: Se requiere un servidor web (por ejemplo, Apache, Nginx) para alojar la aplicación web.
- Lenguajes y Frameworks: El sistema utiliza Java Script para la lógica del lado del servidor y PHP para interactuar con la base de datos. También se pueden utilizar frameworks web como Spring (Java) y Laravel (PHP) para facilitar el desarrollo.
- Base de Datos: El sistema se integra con una base de datos relacional (por ejemplo, MySQL) para el almacenamiento de datos.

3. Instalación y Configuración:

- Proporcione instrucciones detalladas para instalar el servidor web, configurar los lenguajes de programación y establecer la conexión con la base de datos.
- Describa los archivos de configuración relevantes y cómo ajustar los parámetros según las necesidades del entorno.
- Instalación en del servidor en Debían: Primeramente, procederemos a instalar el paquete LAMP Server el cual contiene los siguientes softwares: Apache 2.2.48, Php 8.2, Mariadb Sever-Client y Phpmyadmin (esto para facilitar al usuario). En el caso de Debían nos direccionamos a el menú principal, y luego al terminal allí nos identificamos como superusurio con el comando "su" ingresamos la contraseña y colocamos el siguiente comando "apt install mariadb-server mariadb-client php8.2 libapache2-mod-php8.2 php8.2-mysql php8.2-mongodb" esperamos que se descarguen los paquetes. (Nota: todos los comandos deben escribirse sin comillas).
- Configuración: una vez instalado los paquetes procedemos a saber si el servidor de base de datos se está ejecutando lo hacemos en el terminal, escribimos "systemctl status mariadb", al confirmar que el estatus está activo, procedemos a configurar a mariadb con el comando "mysql" una vez dentro colocamos lo siguiente "alter user root@localhost identified by '123' "esto para decir que el nombre de usuario es root y la contraseña es 123, para salir de mysql ingresamos "quit;". Una vez afuera accedemos a mariadb con el comando "mysql –u root –p" nos pedirá la contraseña e ingresamos la que colocamos anteriomente, escribimos "show databases;" para saber las bases de datos que poseemos y escribimos "quit;" con esto podremos acceder a phpmyadmin. Continuamos creando otro usuario para eso ingresamos a mysql de la misma manera expresada anteriormente y escribimos "grant all privileges on *.* to nombre_de_usuario identified by 'contraseña' with grant option" esto para indicarle que cree un usuario (las partes nombre_de_usuario y contraseña deben ser reemplazadas por el nombre y la contraseña que querramos ponerle a nuestro usuario) luego para darle privilegios administrativos escribimos "flush privileges" y luego "quit" para salir, una vez hecho esto activamos el acceso remoto buscamos el archivo 50-server.cnf en /etc/mysql/mariadb.conf.d/ y comentamos la línea "bind-address 127.0.0.1" con un # al inicio, para después reiniciar el servidor con el siguiente

comando “systemctl restart mariadb, abrimos el cortafuego (si lo estamos utilizando) con “ufw allow mysql” para finalmente acceder remotamente con la siguiente línea “mysql –h 192.168.1.100 –u root –p.

- Instalación del sistema: Para instalar el sistema debemos colocarlo en la carpeta /var/www/html esta es la dirección para que el sistema pueda correr dentro del servidor y modificar el archivo conndb.php (solo si su contraseña de mysql es diferente) , para luego ingresar al navegador e ingresamos: localhost/phpmyadmin , ingresamos el usuario y contraseña de Phpmyadmin y nos situamos en Bases de datos nos ubicamos en importar y en seleccionar archivo lo ubicamos en la carpeta donde está el sistema (su nombre es escuela.sql) ylo abrimos y le damos en continuar y asi tendremos ya nuestra base de datos activa. Para ingresar al sistema en nuestro navegador colocamos la siguiente dirección: localhost/SistemaUEBEM/ y nos redireccionara al index.html que es el login, una vez allí ingresamos nuestro usuario y contraseña.

1. **Estructura del Código Fuente:**

Sistema-UEBEM: contiene el index.php que es la página de login para acceder al sistema y además contiene unas subcarpetas las cuales son:

- Css: que contiene el estilo
- Html: contiene las páginas de los formularios
- Img: contiene las imágenes que se usaron dentro del sistema
- Js: todas las funciones de java script están contenidas dentro de esta carpeta
- Lib: son las librerías que se usaron
- Php: son todas las conexiones que se usaron
- Reportes: en esta carpeta están contenidos todo lo referente al tercer módulo de generación de documentos

2. Seguridad:

1. Autenticación y Autorización: Se implementó un sistema de autenticación seguro que requiere credenciales únicas para cada docente. El acceso al sistema está restringido a usuarios registrados y se asignan roles y permisos adecuados para garantizar que solo el personal autorizado pueda acceder a las funcionalidades correspondientes.
2. Cifrado de Contraseñas: Las contraseñas de los usuarios se almacenan utilizando técnicas de cifrado seguras, como hash. Esto asegura que incluso si la base de datos se ve comprometida, las contraseñas no se puedan recuperar fácilmente.
3. Protección contra Inyecciones SQL: Se aplican técnicas de prevención de inyecciones SQL para evitar que los atacantes manipulen las consultas de la base de datos.
4. Validación de Datos: Se implementa una validación exhaustiva de los datos ingresados por los usuarios para prevenir la introducción de información maliciosa o inválida.
5. Gestión de Sesiones Seguras: Se utiliza una gestión adecuada de sesiones para asegurar que la información de autenticación y autorización se mantenga segura durante la interacción del usuario con el sistema.
6. Control de Acceso Basado en Roles: Se definen roles y privilegios de acceso para los diferentes usuarios del sistema. Esto garantiza que solo aquellos usuarios con los permisos adecuados puedan acceder a ciertas funcionalidades y datos sensibles.

3. Mantenimiento y Actualizaciones:

1. Base de datos:

   - Realizar copias de seguridad periódicas: Establecer una programación para realizar copias de seguridad automáticas de la base de datos de manera regular. Esto puede ser diario, semanal o mensual, dependiendo de la cantidad de datos generados y la importancia de la información almacenada.

- Almacenar las copias de seguridad fuera del servidor principal: Guardar las copias de seguridad en un lugar seguro y separado del servidor principal para proteger los datos en caso de un fallo en el servidor o un ataque.
- Verificar la integridad de las copias de seguridad: Periódicamente, se deben realizar pruebas de restauración de las copias de seguridad para asegurarse de que los datos puedan recuperarse correctamente en caso de ser necesario.

2. Código fuente:
    - Control de versiones: Utilizar un sistema de control de versiones como Git para gestionar y rastrear los cambios en el código fuente. Esto permitirá mantener un historial de las modificaciones realizadas y facilitará la implementación de actualizaciones y parches.
    - Repositorio remoto: Alojar el repositorio del código fuente en un servidor remoto o una plataforma de alojamiento en la nube. De esta manera, se protegerá el código de posibles pérdidas locales o daños en los sistemas.

Cómo realizar actualizaciones y parches del sistema sin afectar su funcionamiento:

1. Entorno de Pruebas: Antes de aplicar una actualización o parche en el ambiente de producción, es recomendable realizar pruebas exhaustivas en un entorno de pruebas. Esto permitirá identificar posibles problemas y corregirlos antes de que afecten el funcionamiento del sistema en producción.
2. Implementación gradual: Si es posible, implementar las actualizaciones y parches de forma gradual. Por ejemplo, se puede liberar primero en un pequeño grupo de usuarios o en una sección específica del sistema antes de extenderlo a todos los usuarios. Esto ayudará a detectar posibles problemas y mitigar su impacto.
3. Realizar copias de seguridad: Antes de realizar cualquier actualización o parche en el entorno de producción, es fundamental realizar una copia de seguridad completa de la base de datos y del código fuente. Esto asegura que, en caso de algún problema durante el proceso, se pueda revertir fácilmente a una versión anterior.
4. Comunicación y Soporte: Informar a los usuarios sobre la actualización o parche y proporcionar soporte para resolver cualquier problema que puedan encontrar.

Mantener una comunicación clara y transparente ayudará a los usuarios a entender los cambios y a adaptarse a las mejoras implementadas.

5. Monitoreo Continuo: Después de aplicar una actualización o parche, es importante realizar un monitoreo constante del sistema para detectar cualquier problema que pueda surgir. Esto permitirá tomar medidas rápidas en caso de que se produzcan problemas inesperados.